

CSS Animations (Анимации)

Введение

CSS Animations позволяют создавать сложные анимации без использования JavaScript. В отличие от переходов (transitions), анимации могут содержать несколько ключевых кадров и автоматически запускаться при загрузке страницы.

Ключевые кадры (@keyframes)

Ключевые кадры определяют состояния анимации в разные моменты времени.

```
@keyframes slide-in {  
  0% {  
    transform: translateX(-100%);  
    opacity: 0;  
  }  
  100% {  
    transform: translateX(0);  
    opacity: 1;  
  }  
}
```

Можно использовать ключевые слова **from** и **to** вместо процентов:

```
@keyframes fade-in {  
  from {  
    opacity: 0;  
  }  
  to {  
    opacity: 1;  
  }  
}
```

Свойства анимации

animation-name

Указывает имя @keyframes анимации, которую нужно применить к элементу.

```
.element {  
  animation-name: slide-in;  
}
```

animation-duration

Задаёт продолжительность одного цикла анимации.

```
.element {  
  animation-duration: 1s;  
}
```

animation-timing-function

Определяет, как анимация прогрессирует в течение одного цикла.

```
.element {  
  animation-timing-function: ease-in-out;  
}
```

Доступные значения: `linear`, `ease`, `ease-in`, `ease-out`, `ease-in-out`, `cubic-bezier()`, `steps()`.

animation-delay

Задержка перед началом анимации.

```
.element {  
  animation-delay: 0.5s;  
}
```

animation-iteration-count

Количество повторений анимации.

```
.element {  
  animation-iteration-count: 3; /* конкретное число */  
}  
  
.infinite-animation {  
  animation-iteration-count: infinite; /* бесконечное повторение */  
}
```

animation-direction

Направление воспроизведения анимации.

```
.element {
  animation-direction: normal; /* по умолчанию */
}
```

Доступные значения:

- **normal**: Анимация воспроизводится вперед каждый цикл
- **reverse**: Анимация воспроизводится в обратном направлении
- **alternate**: Анимация меняет направление в каждом цикле
- **alternate-reverse**: Как alternate, но начинает с обратного направления

animation-fill-mode

Определяет, какие стили применяются до и после выполнения анимации.

```
.element {
  animation-fill-mode: forwards;
}
```

Доступные значения:

- **none**: Элемент возвращается к исходным стилям после анимации
- **forwards**: Элемент сохраняет стили последнего ключевого кадра
- **backwards**: Элемент применяет стили первого ключевого кадра во время задержки
- **both**: Комбинация forwards и backwards

animation-play-state

Управляет воспроизведением анимации.

```
.element {
  animation-play-state: running; /* по умолчанию */
}

.element.paused {
  animation-play-state: paused;
}
```

Сокращенная запись animation

```
.element {
  animation: name duration timing-function delay iteration-count direction
  fill-mode play-state;
}
```

Пример:

```
.element {  
  animation: slide-in 1s ease-out 0.5s 2 alternate forwards;  
}
```

Примеры анимаций

Пульсация

```
@keyframes pulse {  
  0% {  
    transform: scale(1);  
  }  
  50% {  
    transform: scale(1.1);  
  }  
  100% {  
    transform: scale(1);  
  }  
}  
  
.pulse-element {  
  animation: pulse 2s infinite ease-in-out;  
}
```

Вращение

```
@keyframes rotate {  
  from {  
    transform: rotate(0deg);  
  }  
  to {  
    transform: rotate(360deg);  
  }  
}  
  
.spinner {  
  animation: rotate 1.5s linear infinite;  
}
```

Появление с эффектом отскока

```

@keyframes bounce-in {
  0% {
    transform: scale(0);
  }
  60% {
    transform: scale(1.1);
  }
  80% {
    transform: scale(0.9);
  }
  100% {
    transform: scale(1);
  }
}

.bounce-element {
  animation: bounce-in 1s cubic-bezier(0.215, 0.610, 0.355, 1.000);
}

```

Практические советы

1. Используйте анимации умеренно, чтобы не отвлекать пользователей.
2. Для лучшей производительности анимируйте только свойства `transform` и `opacity`.
3. Добавляйте поддержку отключения анимаций для пользователей с вестибулярными расстройствами:

```

@media (prefers-reduced-motion: reduce) {
  * {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}

```

4. Для сложных анимаций рассмотрите использование JavaScript-библиотек, таких как GSAP или Anime.js.
5. Используйте инструменты разработчика в браузере для отладки и оптимизации анимаций.